

SESSION 01

JAVASCRIPT

Variables, Data Types & Basic User Input

VARIABLES

`var / let / const`

DATA TYPES

7 primitive + objects

INPUT

`prompt + Number()`

OUTPUT

`console + typeof`

How to use these notes

Read the short explanation, copy the code by hand, then try the practice task without looking back. Blue handwriting = core note. Yellow boxes = remember. Code blocks = what to type.

Built from your Session 1 JS, HTML and study PDF, with a few missing fundamentals added.

Table of Contents

INDEX

Click a topic to jump there. Use BACK TO INDEX on every page.

03 Source map + HTML connection

18 prompt()

04 console.log()

19 Number() conversion + NaN

05 Variables: declaration + initialization

20 typeof operator

06 var / let / const

21 Complete input program

07 Scope: block vs function

22 Useful related built-ins

08 Hoisting + Temporal Dead Zone

23 Common beginner mistakes

09 Redeclaration vs reassignment

24 Comparison tables

10 const and object/array mutation

25 Practice 1

11 Data types overview

26 Practice 2

12 The 7 primitive types

27 Answer key 1

13 String + Number

28 Answer key 2

14 Boolean + undefined + null

29 Mini project - calculator

15 BigInt + Symbol

30 Quick revision sheet

16 Arrays

31 Clean Session 1 code

17 Objects

32 References + source notes

Study order: understand -> copy -> run -> change -> explain.

Source Map + HTML Connection

Your 3 supplied files and how they fit together.

You gave three inputs:

- s1.js - the original JavaScript source, including variables, data types, arrays, objects, prompt input and Number conversion.
- s1.html - a minimal page that loads s1.js with defer.
- s1(1).pdf - the existing typed study booklet with explanations, tables, practice questions and a calculator mini-project.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Session 1</title>
  <script src="s1.js" defer></script>
</head>
```

Why defer matters

The script is loaded without blocking the HTML parser. The file waits until the document has been parsed before running. For this simple session there is no DOM work yet, but defer is still a clean browser pattern.

Source correction added

Your JS contains `Symbol()`, but the typed PDF listed only six primitive names. JavaScript has seven primitive types: string, number, bigint, boolean, undefined, symbol and null. The handwritten version keeps `Symbol` so the notes match the code.

First Output: console.log()

Start by learning how to see what your program is doing.

```
console.log("Hello");
```

Think of `console.log()` as your temporary notebook inside the browser. It prints values so you can inspect what the program produced.

Remember

The quotes create a string value. `console.log()` itself does not convert a string into a number - it simply displays the value.

Try these:

```
console.log(42);  
console.log(true);  
console.log("JS");  
console.log(10 + 20);
```

Variables: Declaration + Initialization

Two words that look similar but mean different steps.

```
let score;  
score = 50;  
  
// same idea in one line  
let marks = 50;
```

Declaration

Create the binding/name. For let and var, the initializer can be omitted.

Initialization

Give the variable its first value. In let score = 50, the initializer is 50.

Mental model: declaration = create the name; initialization = give it its first value.

var / let / const

Know the rule before memorizing the syntax.

Feature	var	let	const
Can declare without value	Yes	Yes	No
Same-scope redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Scope	Function / global	Block / global	Block / global

Modern rule

Prefer const when you do not need reassignment; use let when reassignment is needed. var is mainly important for understanding older JavaScript and its hoisting behavior.

```
const name = "Roshan";  
let count = 1;  
count = 2;  
// name = "A"; // TypeError
```

Scope: block vs function

This is the missing bridge between the keywords and real code.

Scope answers one question: where can I use this variable?

```
if (true) {  
  let inside = 10;  
  const fixed = 20;  
  console.log(inside, fixed);  
}  
  
// console.log(inside); // ReferenceError
```

Block scope

let and const are limited to the block { ... } in which they are declared. A block can be an if, for, while, or a standalone brace block

var is different

var is function-scoped, not block-scoped. This is one reason modern code usually prefers let and const.

Hoisting + Temporal Dead Zone

A practical explanation, not a slogan.

var declarations are hoisted: the declaration is processed before execution reaches the line. The initializer is not.

```
console.log(a); // undefined
var a = 10;

// roughly think:
var b;
console.log(b); // undefined
b = 10;
```

let and const are also known to be in the scope before their declaration line, but accessing them before the declaration is reached triggers the Temporal Dead Zone (TDZ) and a ReferenceError.

```
console.log(x); // ReferenceError
let x = 10;
```

Exam line

var -> declaration can be observed as undefined before initialization. let/const -> accessing before the declaration line is reached throws due to the TDZ.

Redeclaration vs Reassignment

Do not mix these two ideas in exams.

Operation	Meaning	Example
Redeclaration	Create another declaration with the same name	<code>var x = 1; var x = 2;</code>
Reassignment	Change the current value of an existing binding	<code>let x = 1; x = 2;</code>

```
var x = 70;  
var x = 40; // redeclaration  
x = 66;     // reassignment  
  
let y = 20;  
y = 44;     // reassignment
```

Rule

var permits same-scope redeclaration. let and const do not. var and let can be reassigned; const cannot.

const and Object/Array Mutation

const locks the binding, not every nested property.

```
const user = { name: "Asha" };  
user.name = "Riya"; // allowed  
  
const marks = [10, 20];  
marks.push(30);      // allowed  
  
// user = {};          // error  
// marks = [1, 2];     // error
```

Key idea

const prevents reassignment of the variable binding. It does not make an object or array deeply immutable. Properties/elements can still be changed.

This is why the line `const arr = [1, 2, 3]` is valid even though `arr.push(4)` can later modify the array.

Do not say

"const means the value can never change." A safer sentence is: "const prevents reassignment of the binding."

Data Types Overview

JavaScript is dynamically typed: types belong to values.

For this session, divide the values into two big groups:

- Primitive values: string, number, bigint, boolean, undefined, symbol, null.
- Objects / reference-style values: Array and Object are the examples used in this session. Functions are objects too, but we leave them for later sessions.

Group	Examples	Core idea
Primitive	String, Number, Boolean, Undefined, Null, BigInt, Symbol	Single primitive values
Object	Array, Object	Structured collections / properties

Accuracy note

Avoid memorizing "primitive = stack, object = heap" as if it were the JavaScript language definition. Memory representation is an implementation detail; the language model is about values, references, mutation and semantics.

The 7 Primitive Types

Your JS file contains `Symbol()`, so keep all seven in the notes.

Type	Example	typeof
String	"Hello"	"string"
Number	56 / 12.43	"number"
Boolean	true	"boolean"
Undefined	let x;	"undefined"
Null	null	"object"*
BigInt	123n	"bigint"
Symbol	Symbol("id")	"symbol"

* null is the famous quirk

`typeof null` returns "object" for historical reasons. `null` itself is a primitive value, not an object.

Primitive values are immutable. A variable holding a primitive can still be reassigned - the variable changes to a new value; the existing primitive value is not mutated.

String + Number

Two primitive types you will use constantly.

```
let name = "Roshan";  
let city = "Mumbai";  
  
let a = 56;  
let b = 12.43;  
  
console.log(typeof name); // string  
console.log(typeof a);    // number
```

String

Text values are created with single quotes, double quotes, or template literals. The surrounding quotes are syntax, not part of the value.

Number

JavaScript uses one Number type for integers and floating-point values. There are not separate "int" and "float" types in the JavaScript type system.

Helpful checks

`Number.isInteger(10)` -> true. `Number.isInteger(12.43)` -> false. This is a useful test when your logic needs a whole number.

Boolean + undefined + null

Three values that beginners often mix up.

```
let isReady = true;
isReady = false;

let value;
let empty = null;

console.log(typeof isReady); // boolean
console.log(typeof value);   // undefined
console.log(typeof empty);   // object
```

Value	Common meaning
true / false	Logical state
undefined	No value has been assigned in the usual declaration case
null	Intentional absence / empty value

Easy memory trick

undefined -> JavaScript is telling you the value is not currently defined. null -> the programmer deliberately chose "nothing here".

BigInt + Symbol

The two less-familiar primitives from your JS file.

BigInt

Use BigInt for integers that need precision beyond `Number.MAX_SAFE_INTEGER`. A trailing `n` marks a BigInt literal.

```
const big = 2342323534523453464575686785745n;  
console.log(typeof big); // "bigint"  
  
// do not mix Number and BigInt directly  
// 10n + 5 -> TypeError
```

Symbol

`Symbol()` creates a unique primitive value. It is useful when you need a property key that should not collide with ordinary string keys.

```
const id1 = Symbol("id");  
const id2 = Symbol("id");  
console.log(id1 === id2); // false
```

Arrays

Ordered collections with zero-based indexing.

```
const arr = [1, 2, 3, 4, 5, 6, 6, 7];
```

```
console.log(arr[0]); // 1
```

```
console.log(arr[3]); // 4
```

```
console.log(arr.length); // 8
```

Index rule

The first element is index 0. The last valid index is length - 1.

```
arr.push(8); // add at end
```

```
arr.pop(); // remove last
```

```
arr.includes(4); // true
```

```
Array.isArray(arr); // true
```

Useful methods added

These are not new core topics; they are small tools that naturally belong with arrays. Practice them after you understand indexing.

Objects

Key-value data for representing related information.

```
const obj = {  
  id: 4,  
  name: "Asha",  
  city: "Mumbai"  
};  
  
console.log(obj.name);  
console.log(obj["name"]);
```

Syntax	Meaning
obj.name	Dot notation
obj["name"]	Bracket notation
id: 4	Key-value pair

Useful object helper

`Object.keys(obj)` returns an array containing the object's own enumerable property names. It is handy when you want to inspect which keys exist.

```
console.log(Object.keys(obj));  
// ["id", "name", "city"]
```

prompt()

Browser input: text first.

```
const input = prompt("Enter your name:");  
console.log(input);
```

prompt() opens a browser dialog and returns the user input as a string. If the user cancels the dialog, the returned value is null.

Important

If the user types 10, prompt() gives you the string "10". It is not the number 10 yet.

```
const ageText = prompt("Age?");  
console.log(typeof ageText); // "string"  
  
// cancel -> null  
// do not call Number(null) blindly if cancel matters
```

Number() Conversion + NaN

Turn numeric text into a number, then validate it.

```
Number("10")      // 10
Number("12.5")     // 12.5
Number("10abc")    // NaN
Number("")         // 0
Number(null)       // 0
Number(undefined) // NaN
```

String input + + operator

"10" + "20" gives "1020" because + concatenates strings. Number("10") + Number("20") gives 30 because both operands are numbers.

```
const raw = prompt("Enter a number:");
const value = Number(raw);

if (Number.isNaN(value)) {
  console.log("Invalid number");
}
```

Validation note

Number.isNaN(value) checks specifically whether value is the numeric NaN value. It is usually clearer than relying on global isNaN() when you want a strict check.

typeof Operator

Use it to inspect the type reported for a value.

Expression	Result
typeof "Hello"	"string"
typeof 56	"number"
typeof true	"boolean"
typeof undefined	"undefined"
typeof 123n	"bigint"
typeof Symbol("id")	"symbol"
typeof [] / {}	"object"
typeof null	"object"

What typeof actually returns

It returns a string describing the operand's type category. So typeof 42 is not the word number without quotes; the runtime value is the string "number".

```
console.log(typeof (10 + 20)); // "number"  
console.log(typeof ("10" + "20")); // "string"
```

Complete Input Program

Your original mini-program, explained line by line.

```
let s = Number(prompt("Enter Num1: "));  
let g = Number(prompt("Enter Num2: "));  
  
console.log(  
  "Addition:",  
  s + g,  
  "\nType of Number:",  
  typeof (s + g)  
);
```

- prompt() asks for input.
- Number() converts the text into a number.
- s + g performs numeric addition.
- typeof (s + g) checks the result type.

Flow to memorize

Sample: enter 10 and 20 => Addition: 30, Type of Number: "number".

User input -> prompt() -> string -> Number() -> number -> arithmetic -> result -> typeof -> "number"

Useful Related Built-ins

Small additions that complete the ideas in this session.

Tool	Use it for	Example
<code>Number()</code>	Convert a value to number	<code>Number("25")</code>
<code>String()</code>	Convert a value to string	<code>String(25)</code>
<code>Number.isNaN()</code>	Check for numeric NaN	<code>Number.isNaN(value)</code>
<code>Number.isInteger()</code>	Check whole-number values	<code>Number.isInteger(10)</code>
<code>Array.isArray()</code>	Check whether a value is an array	<code>Array.isArray(arr)</code>
<code>Object.keys()</code>	Get own enumerable keys	<code>Object.keys(obj)</code>

Do not treat these as a separate chapter. They are helper tools you can reach for when working with the same concepts: conversion, validation, arrays and objects.

Study tip

First learn the core syntax. Then add one helper at a time to your practice. The goal is understanding, not collecting names.

Common Beginner Mistakes

Use this page before a practical or viva.

- Writing `const x;` - invalid because `const` must be initialized at declaration time.
- Trying `x = 20` after `const x = 10` - reassignment is not allowed.
- Assuming `prompt()` gives a number - it normally gives a string, or null on cancel.
- Forgetting string concatenation: `"10" + "20" -> "1020"`.
- Confusing redeclaration with reassignment.
- Starting an array at index 1 instead of index 0.
- Thinking `typeof null === "object"` means null is an object - it is a historical quirk.
- Mixing `BigInt` with `Number` in arithmetic without explicit planning.
- Using `var` inside a block and expecting block scope.
- Accessing `let/const` before declaration and forgetting the TDZ.

Fast check

Can you explain each mistake without reading the bullet? If not, circle that topic and practice it again.

Comparison Tables

One-page style revision, split into clean blocks.

Variable rule	var	let	const
No initializer	Yes	Yes	No
Redeclare same scope	Yes	No	No
Reassign	Yes	Yes	No
Scope	Function	Block	Block

Type	Example	typeof
String	"Hi"	string
Number	12.5	number
Boolean	false	boolean
Undefined	let x;	undefined
Null	null	object*
BigInt	123n	bigint
Symbol	Symbol()	symbol
Array	[1,2]	object
Object	{id:1}	object

Three distinctions

Declaration != initialization. Redeclaration != reassignment. String input != numeric value.

Practice 1

Do these without opening the previous pages.

- 1. Print Hello JavaScript with `console.log()`.
- 2. Declare one `var` and one `let` without a value.
- 3. Write `const value = 25` correctly.
- 4. Explain declaration vs initialization.
- 5. Show a valid `var` redeclaration.
- 6. Show a valid `let` reassignment.
- 7. Why is `const x;` invalid?
- 8. Create a string variable called `course`.
- 9. Create one integer-like number and one decimal number.
- 10. Create a Boolean and change `true` to `false`.
- 11. What value does an uninitialized `let` variable have?

Practice 2

Concept and prediction questions.

- 12. Create an array of five numbers and print the first element.
- 13. Create an object with id and name; access name with dot notation.
- 14. Give one redeclaration and one reassignment example.
- 15. Create a BigInt literal. What does n mean?
- 16. Explain undefined vs null.
- 17. Predict "10" + "20".
- 18. Predict Number("10") + Number("20").
- 19. What does typeof true return?
- 20. What does typeof null return and why?
- 21. Which index contains 72 in [45,60,72]?
- 22. Write a two-number addition program using prompt, Number and typeof.

Answer Key 1

Self-check after Practice 1.

- 1. `console.log("Hello JavaScript");`
- 2. `var a; let b;`
- 3. `const value = 25;`
- 4. Declaration creates the binding; initialization gives it the first value.
- 5. `var x = 1; var x = 2;`
- 6. `let x = 1; x = 2;`
- 7. `const` requires an initializer immediately.
- 8. `let course = "JavaScript";`
- 9. `let a = 10; let b = 12.5;`
- 10. `let ready = true; ready = false;`
- 11. `undefined.`

Answer Key 2

Self-check after Practice 2.

- 12. `const nums = [10,20,30,40,50]; console.log(nums[0]);`
- 13. `const user = { id: 1, name: "Asha" }; console.log(username);`
- 14. Redclaration: `var x = 1; var x = 2;` Reassignment: `x = 3;`
- 15. `const big = 12345678901234567890n;` n marks a BigInt literal.
- 16. undefined commonly means no value has been assigned; null is an intentional absence value.
- 17. "1020" because + concatenates strings.
- 18. 30 because `Number()` converts the text first.
- 19. "boolean".
- 20. "object" - historical JavaScript behavior for null.
- 21. Index 2.
- 22. Example: `let a = Number(prompt("A?")); let b = Number(prompt("B?")); console.log(a+b, typeof(a+b));`

Mini Project - Simple Calculator

Put the session together in one browser exercise.

Task: ask the user for two numbers and print addition, subtraction, multiplication and division. Also show the type of one numeric result.

```
const a = Number(prompt("Enter first number:"));
const b = Number(prompt("Enter second number:"));

console.log("Add:", a + b);
console.log("Subtract:", a - b);
console.log("Multiply:", a * b);
console.log("Divide:", a / b);
console.log("Type:", typeof (a + b));
```

Extension challenge

Handle cancel and invalid numbers. Then use `Number.isNaN()` to decide when to show an error instead of a result.

Flow

Input -> convert -> validate -> calculate -> display -> inspect type

Quick Revision Sheet

One page to read before a quiz, viva or practical.

Variables

var -> function-scoped + redeclarable; let -> block-scoped + reassignable; const -> block-scoped + no reassignment; const needs initialization.

Value flow

Declaration -> initialization -> reassignment (when allowed). Redeclaration is a separate operation.

Types

7 primitives: string, number, bigint, boolean, undefined, symbol, null. Arrays/objects are objects.

Input

prompt() -> string or null. Number() -> convert. Number.isNaN() -> strict NaN check.

Inspection

typeof value -> returns a string describing the type category. typeof null -> "object".

Arrays

Index starts at 0. Use length, push, pop, includes, Array.isArray as beginner helpers.

Objects

key:value pairs. Use dot/bracket notation. Object.keys(obj) lists keys.

Accuracy

const prevents reassignment, not object mutation. Do not reduce JS memory to stack vs heap slogans.

Clean Session 1 Code

Normalized version of the original JS, plus safe comments.

```
console.log("Hello");

// Variables
var a;
let b;
const c = 10;

a = 10;
b = 20;

var x = 70;
let y = 20;
const z = 43;

var x = 40; // redeclaration
x = 66;     // reassignment
y = 44;

// Data types
let str = "Hello";
let num = 56;
let num1 = 12.43;
let isReady = true;
isReady = false;
let un;
let nullVal = null;
let bigNum = 2342323534523453464575686785745n;
let sym = Symbol("id");

// Array
const arr = [1, 2, 3, 4, 5, 6, 6, 7];

// Object
const obj = { id: 4, key: 78 };

// User input
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));
console.log("Addition:", s + g);
console.log("Type:", typeof (s + g));
```

Note

This page stays close to your original code. The earlier pages explain the missing fundamentals around it instead of changing the basic session order.

References + Source Notes

What was used to build these handwritten notes.

Primary user-provided sources:

- s1.js - original session code and comments.
- s1.html - browser page that loads s1.js with defer.
- s1(1).pdf - existing 35-page Session 1 study booklet and its visual structure.

Reference documentation consulted for missing / clarified fundamentals:

- MDN JavaScript - Variables and declarations:
developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types
- MDN JavaScript - Data types:
developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures
- MDN JavaScript - typeof:
developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof
- W3Schools JavaScript - JavaScript reference and basics: [w3schools.com/js/](https://www.w3schools.com/js/)

What was added

Symbol, block/function scope, TDZ, var hoisting behavior, const mutation, prompt cancellation, Number.isNaN, Number.isInteger, Array.isArray and Object.keys were added/clarified where they support the same Session 1 concepts.

Typography

The notebook content is intentionally rendered in a handwriting-like style, while clean system-style typography is used for small navigation and reference labels.